VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Asignatura
Analisis de algoritmo

Presentado por:
Melissa Restrepo Hurtado
Alfonso Ocampo Montealegre
Camila Ramirez Galvis

Docente:
Sergio Augusto Cardona Torres



Universidad del Quindio
2026

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

Tabla de contenido

Fuente de Información y Preprocesamiento....3
Corpus....3
Preprocesamiento del Texto....3
Palabras Eliminadas....6
Comparación del Ranking sin y con Preprocesamiento....7
Limitaciones del Preprocesamiento....8
Construcción de la Lista de Frecuencias....9
Parámetros del Corpus....9
Arquitectura de la Solución....10
Descripción de los Algoritmos....12
Counting Sort....13
Radix Sort....15
Bucket Sort....17
Fundamentación Teórica....20
Cota Inferior de los Algoritmos por Comparación....21
Cómo Evitan la Cota los Algoritmos No Comparativos....22
Análisis de Counting Sort....22
Análisis de Radix Sort....23
Análisis de Bucket Sort....24
Síntesis del Análisis Teórico....25
Implementación....26
Decisiones de Implementación....26
Metodología....26
Resultados sobre el Corpus....27
Análisis de los Resultados....29
Experimento Complementario de Escalabilidad....30
Validación de Resultados....33
Tratamiento de los Empates....35
Interpretación del Ranking....36
Pruebas de Verificación....36
Uso de la Aplicación....37
Conclusiones....37
Declaración de Uso de Inteligencia Artificial....38
Anexos....39

Ranking de Términos mediante Algoritmos de Ordenamiento No Comparativos sobre un Corpus de 20 Artículos Científicos

Este documento presenta el desarrollo del seguimiento *Ranking de términos mediante algoritmos de ordenamiento*, del curso Análisis de Algoritmos. A partir del preprocesamiento de los abstracts de 20 artículos científicos sobre inteligencia artificial generativa en educación, la aplicación genera un listado de los términos más frecuentes del corpus, ordenado con tres algoritmos que no se basan en comparaciones directas entre elementos: Counting Sort, Radix Sort y Bucket Sort.

El enunciado del seguimiento exige tres análisis, que se resumen en la Tabla 1. Los tres algoritmos se implementaron desde cero en Java, sin usar funciones de ordenamiento de la librería estándar (Arrays.sort, Collections.sort o List.sort), conforme a la restricción del curso de no usar funciones de alto nivel que implementen directamente los algoritmos solicitados.

Tabla 1

Entregables del Seguimiento y Sección donde se Desarrollan

Entregable	Qué exige el enunciado	Sección
Fundamentación teórica	Explicar por qué los tres algoritmos logran complejidades O(n + k) u O(n)	Fundamentación Teórica
Medición empírica	Ejecutar los tres algoritmos sobre la lista de frecuencias del corpus completo y medir	Medición Empírica

	los tiempos reales en milisegundos	
Validación de resultados	Presentar una tabla o gráfico de los 15 términos más frecuentes obtenidos con cada algoritmo	Validación de Resultados

Fuente: Elaboración propia

Fuente de Información y Preprocesamiento

Corpus

La fuente de información son los 20 artículos científicos en formato PDF entregados por el docente. Como lo indica el enunciado, el procesamiento se hace exclusivamente sobre el abstract de cada artículo. Los abstracts se extrajeron una sola vez de los PDF, se revisaron manualmente y se guardaron en el archivo `artículos.json`, que es la fuente que lee la aplicación por medio de la clase `Repositorio Artículos`.

Preprocesamiento del Texto

Cada abstract pasa por el método Preprocesador.procesarParaEmbeddings, implementado desde cero, que aplica tres etapas:

1. **Normalización:** conversión a minúsculas y eliminación de tildes y signos de puntuación; se conservan letras, dígitos y espacios.
2. **Tokenización:** separación del texto en palabras. Se descartan los tokens formados solo por dígitos, como años o porcentajes.

3. **Eliminación de *stopwords*:** se quitan las palabras vacías del inglés (*the*, *of*, *and*, *is*) y conectores académicos que no aportan significado (*however*, *study*, *paper*, *review*), con una lista propia de 191 palabras. También se descartan las palabras de una sola letra.

No se aplica *stemming* (reducción a la raíz), para que el ranking muestre palabras reales del idioma, como education o learning, y no raíces truncadas como educ o learn, a continuación se mostrará por medio de la tabla 2 como se aplicó el preprocesamiento en los abstract tomando como ejemplo el primer párrafo del primer artículo.

Tabla 2

Efecto de cada Etapa del Preprocesamiento sobre una Oración del Corpus

Etapa	Resultado
Texto original	Since artificial intelligence (AI) is finding its way into more and more areas of everyday life, improving the AI skills of non-experts is important and will become even more relevant in the future.
1. Normalización	since artificial intelligence ai is finding its way into more and more areas of everyday life improving the ai skills of non experts is important and will become even more relevant in the future
2. Tokenización	34 tokens: [since, artificial, intelligence, ai, is, finding, its, way, into, more, and, more, areas, of, everyday, life, improving, the, ai, skills, of, non, experts, is, important, and, will, become, even, more, relevant, in, the, future.]
3. Sin stopwords	20 tokens: [since, artificial, intelligence, ai, finding, way, areas, everyday, life, improving, ai, skills, non, experts, important, will,

	become, even, relevant, future]
Palabras eliminadas	is, its, into, more, and, more, of, the, of, is, and, more, in, the (14)

Fuente: Elaboración propia.

Efecto sobre el Corpus Completo

La Tabla 3 y la Figura 1 resumen cuántas palabras quedan después de cada etapa en los 20 abstracts. De las 3978 palabras que produce la tokenización, la eliminación de *stopwords* descarta 1567, es decir, el 39.4 %, y las palabras de una sola letra descartan otras 9. Las 2402 palabras resultantes, con 980 términos distintos, son exactamente la entrada de los algoritmos de ordenamiento descrita en la Tabla 4, lo que se puede verificar con la operación 3978 − 1567 − 9 = 2402.

Tabla 3

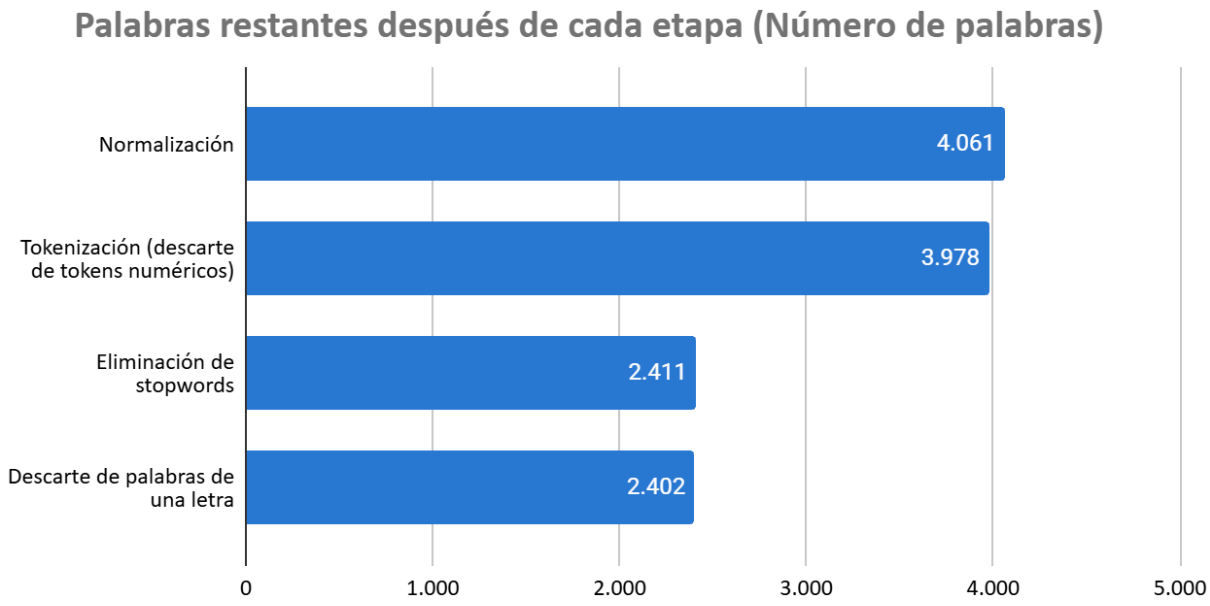
Palabras del Corpus Después de cada Etapa del Preprocesamiento

Etapa	Palabras eliminadas	Palabras restantes	Términos distintos
Normalización	-	*4061*	-
Tokenización (descarte de tokens numéricos)	*83*	*3978*	*1070*
Eliminación de stopwords	*1567*	*2411*	-
Descarte de palabras de una letra	*9*	*2402*	*980*

Fuente: *Elaboración propia*

Figura 1

Número de Palabras del Corpus Después de cada Etapa del Preprocesamiento



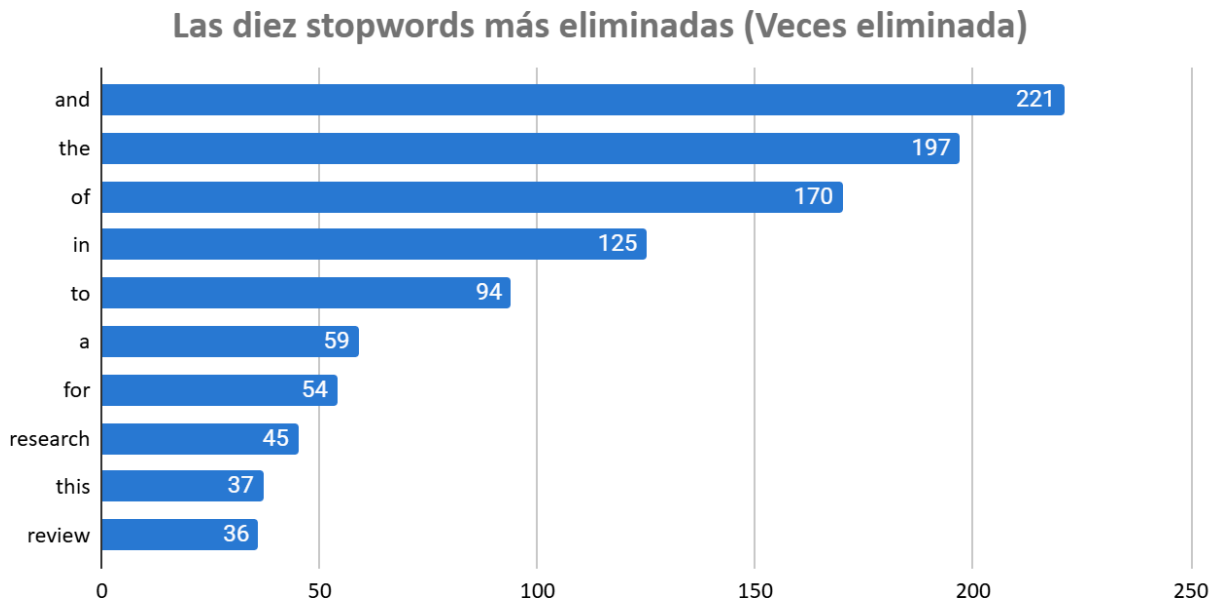
Fuente. Elaboración propia.

Palabras Eliminadas

La Figura 2 presenta las diez *stopwords* que más veces se eliminaron. Las cinco primeras (*and*, *the*, *of*, *in* y *to*) son conectores y artículos del inglés que suman 807 apariciones, más de la mitad (51.5 %) de todo lo que se eliminó. También aparecen *research* y *review*, que no son palabras vacías del idioma, pero se incluyeron en la lista porque en un corpus de revisiones de literatura aparecen en casi todos los abstracts y no ayudan a distinguir su contenido.

Figura 2

Las Diez Stopwords más Eliminadas del Corpus



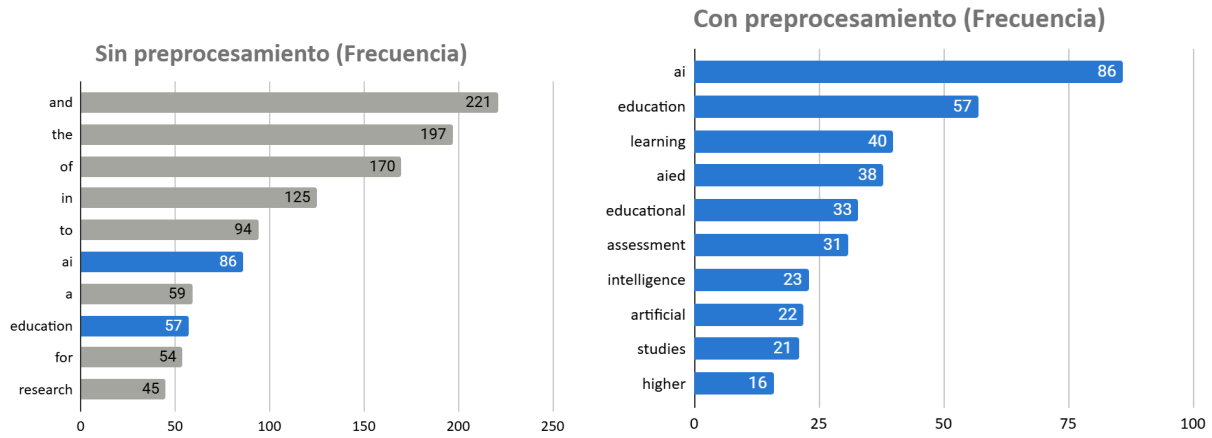
Fuente: *Elaboración propia*

Comparación del Ranking sin y con Preprocesamiento

La Figura 3 muestra por qué el preprocesamiento es indispensable para el ranking. Sin eliminar las *stopwords*, ocho de los diez términos más frecuentes son palabras sin contenido (*and*, *the*, *of*, *in*, *to*, *a*, *for* y *research*), y solo *ai* y *education* describen el tema del corpus. Con el preprocesamiento, los diez términos tienen significado y permiten reconocer de inmediato el dominio de los artículos: la inteligencia artificial aplicada a la educación.

Figura 3

Los Diez Términos más Frecuentes del Corpus sin y con Preprocesamiento



Fuente: Elaboración propia

Limitaciones del Preprocesamiento

En el ejemplo de la Tabla 2 sobreviven palabras poco informativas como *since*, *will*, *even* y *become*, porque no están en la lista de *stopwords*. Estas palabras aparecen pocas veces y no alteran el top 15, pero la lista se podría ampliar para obtener un resultado más limpio. Además, como no se aplica *stemming*, formas relacionadas como *education* y *educational* se cuentan por separado; esta decisión se tomó para que el ranking muestre palabras reales, a cambio de dividir la frecuencia de algunos conceptos. Los datos con los que se construyeron las Figuras 1, 2 y 3, junto con sus cálculos y las gráficas en formato editable, se presentan en el Anexo 1: Graficas del Preprocesamiento, para que el lector pueda consultar o verificar cada valor.

Construcción de la Lista de Frecuencias

La clase `ContadorFrecuencias` recorre los tokens de los 20 abstracts y cuenta cuántas veces aparece cada término. El resultado es un arreglo de pares TerminoFrecuencia(termino, frecuencia), en el que la frecuencia es la clave de ordenamiento, un entero entre 1 y k, y el término es el dato satélite que viaja con ella. Los algoritmos mueven el par completo, no solo el número.

El conteo usa un mapa que conserva el orden de primera aparición de cada término (artículo 1, 2, …, 20). Ese orden es la entrada de los tres algoritmos; sus primeros elementos son since = 1, artificial = 22, intelligence = 23, ai = 86, finding = 1 y way = 2.

Parámetros del Corpus

La Tabla 4 resume los valores del corpus que intervienen en el análisis de complejidad. Es importante no confundir n = 980, que es el número de palabras distintas, con las 2402 palabras en total: el término *ai* aparece 86 veces, pero es un solo elemento de la lista que se ordena.

Tabla 4

Parámetros del Corpus que Intervienen en el Análisis

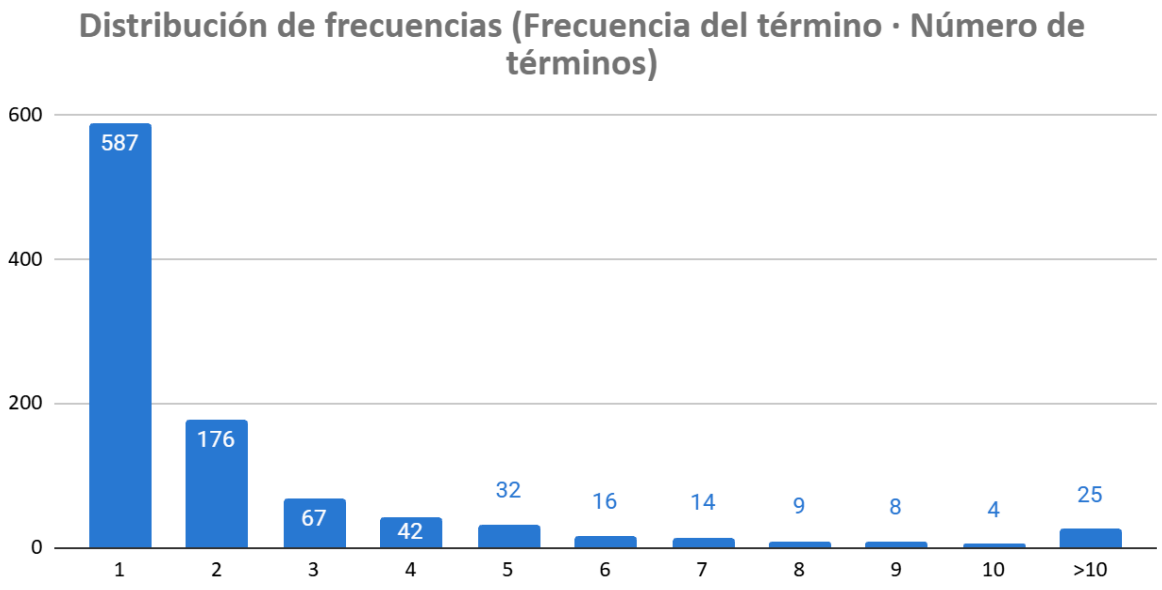
Parámetro	Símbolo	Valor	Significado
Términos distintos	n	980	Tamaño de la entrada de los algoritmos
Total de palabras (tokens)	–	2402	Suma de todas las frecuencias
Frecuencia máxima	k	86 (*ai*)	Rango de claves de Counting Sort

Frecuencia mínima	–	1	Límite inferior del rango de claves
Valores de frecuencia distintos	–	25	Cubetas ocupadas en Bucket Sort
Dígitos de k en base 10	d	2	Pasadas de Radix Sort
Cubetas de Bucket Sort	m	980 (= n)	Cantidad de cubetas creadas

Fuente: Elaboración propia

Figura 4

Distribución de las Frecuencias de los 980 Términos del Corpus



Fuente: *Elaboración Propia*

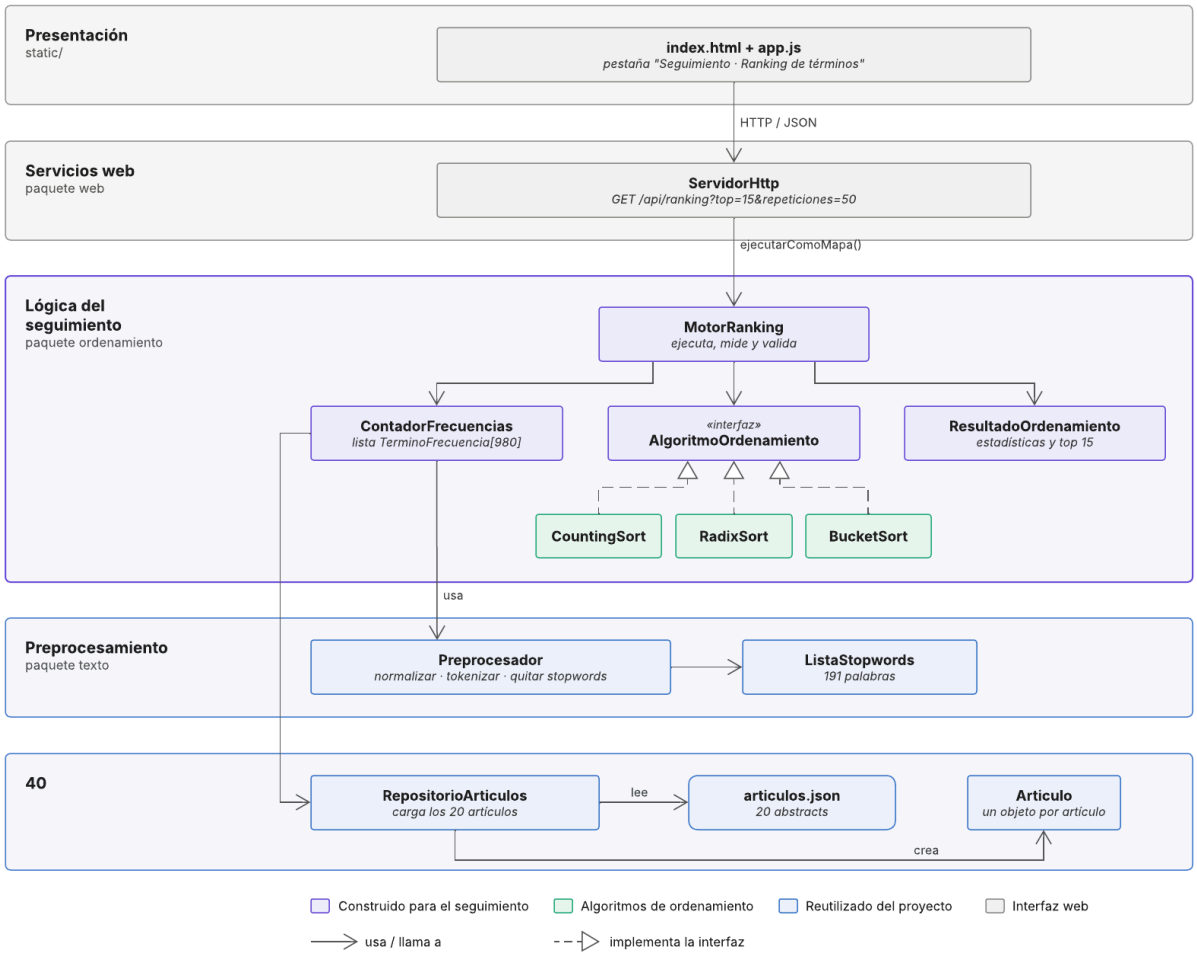
Arquitectura de la Solución

La Figura 5 muestra la Arquitectura por capas en la que cada capa sólo utiliza los servicios de las capas inferiores, presentamos las 5 capas del sistema y los componentes que

intervienen en el ranking de términos. Se presenta en el Anexo 2: Arquitectura en capas para una mejor visualización.

Figura 5

Arquitectura por Capas del Ranking de Términos



Fuente: *Elaboración propia*

Nota: Las flechas continuas indican que un componente usa o llama a otro; las líneas discontinuas con triángulo indican la implementación de una interfaz.

Los tres algoritmos cumplen dos propiedades que se usan en todo el análisis. La primera es el orden descendente: la frecuencia mayor queda primero. La segunda es la estabilidad: si dos términos tienen la misma frecuencia, conservan el orden que tenían en la entrada. Gracias a la estabilidad, los tres algoritmos producen exactamente el mismo ranking, incluso en los empates, y se pueden validar unos contra otros.

Descripción de los Algoritmos

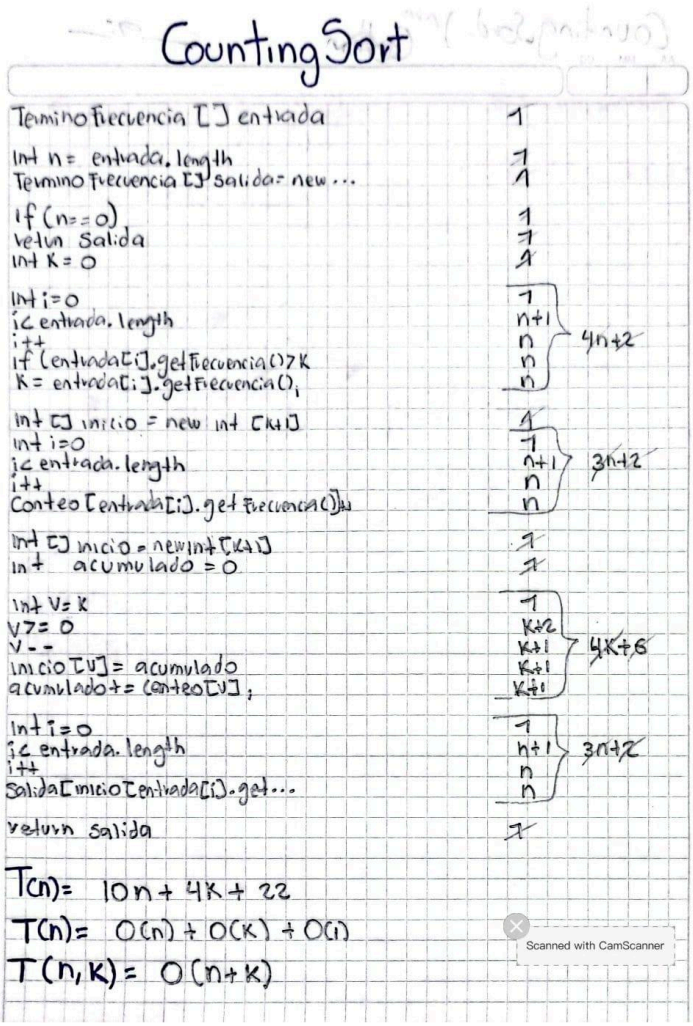
Para determinar la complejidad de cada algoritmo se calculó manualmente su tiempo de ejecución T, contando las veces que se ejecuta cada línea del código implementado. Se asignó costo 1 a cada asignación, comparación, operación aritmética, acceso a un arreglo y retorno; la condición de un ciclo se contó una vez más que el número de vueltas, porque la última evaluación es la que termina el ciclo, y el cuerpo se contó tantas veces como vueltas da el ciclo. A partir de la expresión obtenida se eliminaron las constantes y los coeficientes para llegar al orden de complejidad.

Counting Sort

Counting Sort ordena los términos sin compararlos entre sí. Primero recorre la lista para encontrar la frecuencia máxima *k*; luego cuenta cuántos términos tienen cada frecuencia en un arreglo de $k + 1$ posiciones; después calcula, recorriendo las frecuencias de mayor a menor, la posición de la salida en la que debe empezar cada valor; y finalmente recorre la lista original y ubica cada término directamente en su posición. La Figura 6 presenta el conteo línea por línea del método ordenarDescendente.

Figura 6

Tiempo de ejecución del algoritmo Counting Sort



Fuente: Elaboración propia

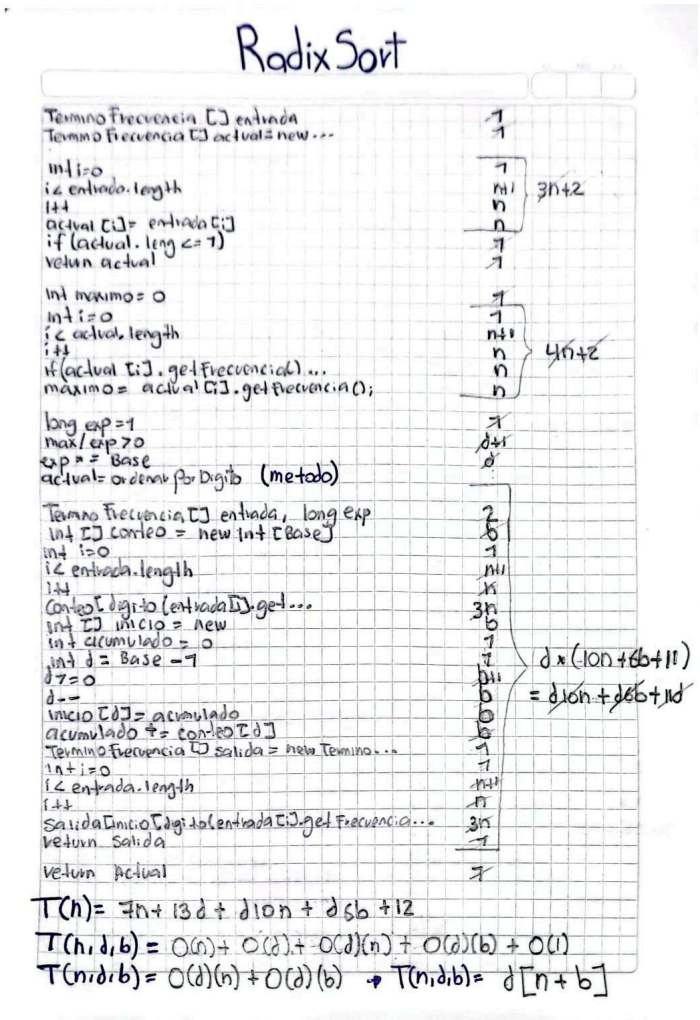
Los ciclos que recorren la lista de términos dependen de n, mientras que el ciclo que calcula las posiciones de inicio se repite $k + 1$ veces, porque su variable va desde k hasta 0. Al sumar todos los bloques se obtiene $T(n, k) = 10n + 4k + c$, donde c agrupa las constantes. Al eliminar coeficientes y constantes, $T(n, k) \in O(n) + O(k) + O(1) = O(n + k)$. Como en el corpus $k = 86$ es menor que $n = 980$, el término n domina y el algoritmo se comporta como $O(n)$.

Radix Sort

Radix Sort ordena las frecuencias dígito por dígito, empezando por las unidades y continuando con las decenas. En cada pasada aplica un Counting Sort estable que solo tiene en cuenta el dígito actual, con diez valores posibles (del 0 al 9). Como cada pasada conserva el orden de la anterior cuando dos frecuencias empatan en un dígito, al terminar todas las pasadas la lista queda ordenada por la frecuencia completa. El número de pasadas d es el número de dígitos de la frecuencia máxima. La Figura 7 presenta el conteo línea por línea de los métodos ordenarDescendente y ordenarPorDigito.

Figura 7

Tiempo de ejecución del algoritmo Radix Sort



Fuente: Elaboración propia

Cada pasada de ordenarPorDigito recorre dos veces la lista de n términos y una vez un arreglo de b = 10 posiciones, y el método se ejecuta d veces, una por cada dígito de la frecuencia máxima. Al sumar todos los bloques se obtiene T(n, d, b) = 10dn + 6db + 7n + 13d + 12. Como d ≥ 1, el término 7n queda acotado por 7dn, y como b es una constante, el término 13d queda acotado por

13db; por lo tanto, $T(n, d, b) \in O(d \cdot (n + b))$. En el corpus, $d = 2$ y $b = 10$ son constantes, por lo que el algoritmo se comporta como $O(n)$: para $n = 980$ la expresión da 26 558 operaciones.

Bucket Sort

Bucket Sort ordena los términos repartiéndolos en cubetas según su frecuencia, en lugar de compararlos entre sí. Su implementación tiene tres métodos. El método distribuir busca la frecuencia mínima y la máxima, crea m = n cubetas vacías y envía cada término a la cubeta que le corresponde, de modo que las frecuencias mayores quedan en las cubetas de índice mayor. Luego, el método ordenarDescendente recorre las cubetas desde la última hasta la primera, ordena cada una con el método ordenarPorInsercionDescendente y copia sus términos a la lista de salida. Como el costo de la Inserción depende de cuántos términos tiene cada cubeta y de cuánto se deben desplazar, el conteo utiliza dos variables adicionales: s, el número de términos de cada cubeta, y D, el número de desplazamientos que hace la Inserción dentro de ella. La Figura 8 presenta el conteo línea por línea de los tres métodos.

Figura 8

Tiempo de ejecución del algoritmo Bucket Sort

BucketSort

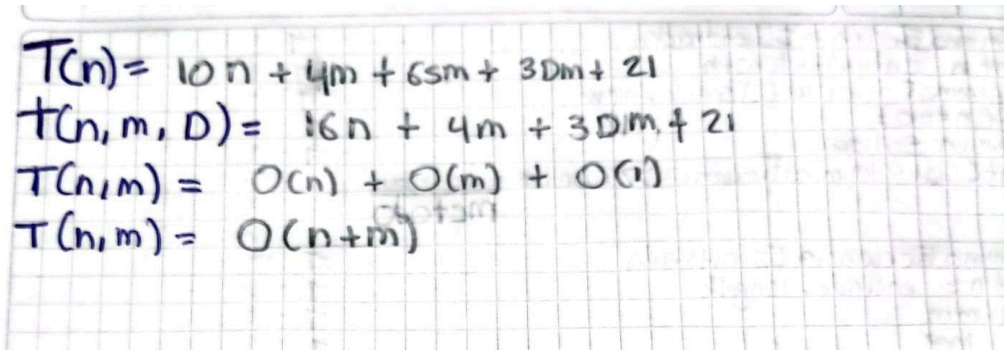
```
terminoFrecuencia[] entrada                                  1
int n = entrada.length                                       1
terminoFrecuencia[] salida = new                             1
if(n==0)                                                     1
return Salida                                                1
List<List<terminoFrecuencia>> cubetas = distribuir()
                                  metodo

TerminoFrecuencia[] entrada                                  1
int n = entrada.length                                       1
int min                                                      1
int max                                                      1
for(TerminoFrecuencia t : entrada)                           n+1
if(t.getFrecuencia() < min)                                  n
min = t.getFrecuencia()                                      n
if(t.getFrecuencia() > max)                                  n
max = t.getFrecuencia();                                     n
int m = n                                                    1
long rango = (long) max - min + 1                            1
List<List<TerminoFrecuencia>> cubetas = ...                  1
int i = 0                                                    1
i < m                                                        m+1
i++                                                          m
cubetas.add(new ArrayList<>())                               m
for(TerminoFrecuencia t : entrada)                           n+1
int indice = (int) ((t.getFrecuencia() - min...              n
Cubetas.get(indice) ...                                      n
return cubetas                                               1
                                                                  = 8n + 3n + 12
int pos = 0                                                  1
int i = cubetas.size()-1                                     1
i >= 0                                                       m+1
i--                                                          m
List<TerminoFrecuencia> cubeta                               m
Ordenar por Insercion Descendente(cubeta)
                                  metodo
List<TerminoFrecuencia> cubeta                               1
int i = 1                                                    1
i < cubeta.size();                                           s
i++                                                          s-1
TerminoFrecuencia actual = cubeta.get...                     s-1
int j = i-1                                                  s-1
while(j >= 0 && cubeta.size...                               Σ(ti+1) = D+(s-1)
cubeta.set(j+1, cubeta.get(j));                              Σti = D
j--                                                          Σti = D
cubeta.set(j+1, actual);                                     s-1
for(TerminoFrecuencia t : cubeta                             n+m
Salida[pos++] = t                                            n
return Salida                                                1
```

= 5s + 3D + (s-1) - 2

= (6s + 3D - 3) * m

= 6sm + 3Dm - 3m



Fuente: Elaboración propia

El conteo línea por línea da como resultado T(n, m, s, D) = 10n + 4m + 6sm + 3Dm + 21, donde s es el número de términos de cada cubeta y D el número de desplazamientos que hace la Inserción dentro de ella, suponiendo cubetas de igual tamaño. Como entre todas las cubetas suman los n términos, s · m = n y la expresión se reduce a T(n, m, D) = 16n + 4m + 3Dm + 21. Todos los términos son lineales excepto 3Dm, cuyo valor depende de cómo quedan repartidos los datos, por lo que el análisis distingue tres casos. En el caso esperado, cuando las frecuencias se reparten de manera uniforme entre m = n cubetas, cada cubeta recibe en promedio un término, el valor esperado de D es menor que 1 y E[T] < 16n + 7m + 21, que pertenece a O(n + m). En el peor caso, si todos los términos caen en una sola cubeta (m = 1, s = n) en orden inverso, D = n(n − 1)/2 y T = 1.5n² + 14.5n + 25, que pertenece a O(n²). En el corpus, cada cubeta contiene términos con la misma frecuencia, por lo que la Inserción no hace desplazamientos (D = 0); con m = n = 980 se obtiene T = 20n + 21 = 19 621 operaciones, es decir, O(n).

Fundamentación Teórica

El análisis de complejidad utiliza tres notaciones asintóticas que describen cómo crece el tiempo de ejecución T(n) de un algoritmo a medida que aumenta el tamaño de la entrada, sin tener en cuenta las constantes ni los términos de menor orden. La Tabla 5 resume su significado. En este documento se usa principalmente O, porque el enunciado pide justificar complejidades de la forma O($n + k$) u O(n); Ω se usa para expresar la cota inferior de los algoritmos por

comparación, y Θ cuando el crecimiento está acotado por arriba y por abajo por la misma función.

Tabla 5

Notaciones Asintóticas Utilizadas en el Análisis

Notación	Nombre	Significado	Ejemplo en este documento
$T(n) \in O(g(n))$	O grande	Cota superior: $T(n)$ no crece más rápido que $g(n)$	Counting Sort: $10n + 4k + c \in O(n + k)$
$T(n) \in \Omega(g(n))$	Omega	Cota inferior: $T(n)$ no crece más lento que $g(n)$	Todo ordenamiento por comparación es $\Omega(n \log n)$
$T(n) \in \Theta(g(n))$	Theta	Cota ajustada: $T(n)$ está en $O(g(n))$ y en $\Omega(g(n))$ a la vez	$\log_2(n!) = \Theta(n \log n)$

Fuente: Elaboración propia

Cota Inferior de los Algoritmos por Comparación

Un algoritmo que solo obtiene información comparando pares de elementos, como Quicksort, Mergesort, Heapsort o Inserción, se puede modelar con un árbol de decisión binario (Cormen et al., 2009). Cada nodo interno es una comparación $a[i] \leq a[j]$ con dos salidas, cada hoja es una de las $n!$ permutaciones posibles de la entrada, y el número de comparaciones en el peor caso es la altura h del árbol. Para funcionar con cualquier entrada, el árbol debe tener al menos $n!$ hojas, y un árbol binario de altura h tiene como máximo 2^h hojas; por lo tanto:

$$2^h \geq n! \quad \Rightarrow \quad h \geq \log_2(n!) = \Theta(n \log n)$$

Es decir, cualquier ordenamiento por comparación hace $\Omega(n \log n)$ comparaciones en el peor caso. Para el corpus, $\log_2(980!) \approx 8\,300$ comparaciones como mínimo.

Cómo Evitan la Cota los Algoritmos No Comparativos

Counting Sort, Radix Sort y Bucket Sort no deciden la posición de un elemento comparándolo con otro: usan el valor de la clave como índice de un arreglo. En el modelo de computación RAM, acceder a una posición de un arreglo cuesta O(1) sin importar cuál sea, y como estos algoritmos no son de comparación, el árbol de decisión no los describe y la cota $\Omega(n \log n)$ no se les aplica.

El precio es una suposición sobre los datos: las claves deben ser enteros en un rango acotado $[0, k]$ o, en el caso de Bucket Sort, repartirse de manera aproximadamente uniforme. Las frecuencias de términos cumplen la primera condición, porque son enteros entre 1 y $k = 86$.

Análisis de Counting Sort

Al contar línea por línea las operaciones del código Tabla 6, los ciclos que recorren la entrada dependen de n, mientras que el ciclo que calcula las posiciones de inicio se repite $k + 1$ veces, porque la variable v va desde k hasta 0. La suma da como resultado:

$$T(n, k) = 10n + 4k + c \quad \in \quad O(n) + O(k) + O(1) = O(n + k)$$

Tabla 6

Conteo de Operaciones de Counting Sort por Bloque

Bloque del código	Costo
Instrucciones iniciales y retorno	constante
Ciclo que busca la frecuencia máxima k	4n + 2
Ciclo de conteo	3n + 2
Ciclo de posiciones de inicio (de k hasta 0)	4k + 6
Ciclo que ubica cada elemento en la salida	3n + 2
Total	10n + 4k + c

Fuente: Elaboración propia

Nota: c agrupa todas las constantes. Si la creación de los arreglos de k + 1 posiciones se cuenta con costo k + 1, el resultado es 10n + 6k + c, que también pertenece a O(n + k).

El espacio adicional es O($n + k$), por los arreglos de conteo, de inicio y de salida. Si k = O(n), el término k no domina y el algoritmo es O(n). En el corpus, $k = 86$ y $n = 980$: el arreglo de conteo tiene solo 87 posiciones y el costo lo domina n. En cambio, si k fuera muy grande respecto a n, como al ordenar diez números entre 0 y 10^9, el arreglo de conteo sería enorme y el algoritmo dejaría de ser eficiente.

Análisis de Radix Sort

Cada pasada de Radix Sort es un Counting Sort sobre un dígito con $b = 10$ valores posibles, de costo O($n + b$), y el número de pasadas es el número de dígitos de la clave máxima, $d = \lfloor \log_{10} k \rfloor + 1$. Por lo tanto:

$$T(n) = d \cdot (c \cdot n + c' \cdot b) \quad \in \quad O(d \cdot (n + b))$$

En el corpus, $b = 10$ y $d = 2$ son constantes, así que $T(n) \in O(n)$. En general, Radix Sort es lineal siempre que las claves tengan un número acotado de dígitos.

La corrección se demuestra por inducción sobre las pasadas. La hipótesis es que, después de la pasada i, el arreglo está ordenado según los i dígitos menos significativos. En la pasada $i + 1$, dos claves con distinto dígito quedan en el orden correcto por ese dígito; si tienen el mismo dígito, la estabilidad del Counting Sort conserva el orden que ya tenían por los dígitos anteriores. Al terminar las d pasadas, el arreglo está ordenado por la clave completa.

Análisis de Bucket Sort

Al contar línea por línea los tres métodos de la implementación (`ordenarDescendente`, `distribuir` y `ordenarPorInsercionDescendente`), con n_i como el número de elementos de la cubeta i, q como el número de cubetas no vacías y D como el total de desplazamientos que hace la Inserción, se obtiene, con $m = n$:

$$T(n) = 25n - 6q + 3D + 18$$

Toda la parte que no depende de D es lineal; el costo solo puede crecer más rápido por los desplazamientos de la Inserción, y estos dependen de cómo quedaron repartidos los elementos. Por eso se analizan tres casos:

- **Peor caso.** Si las n claves, distintas y en orden creciente, caen en una sola cubeta ($q = 1$), entonces $D = n(n - 1)/2$ y $T(n) = 1.5n^2 + 23.5n + 12$, que pertenece a $O(n^2)$.

- **Caso esperado.** Si las claves se reparten de manera uniforme entre $m = n$ cubetas, el valor esperado del cuadrado del tamaño de cada cubeta es $E[n_i^2] = 2 - 1/n$ (Cormen et al., 2009); el costo esperado de la Inserción por cubeta queda acotado por una constante y $E[T(n)] \in O(n + m) = O(n)$.
- **Caso del corpus.** La distribución de Zipf no es uniforme: la cubeta de la frecuencia 1 recibe 587 términos. Sin embargo, como el rango de claves (86) es menor que $m = 980$, cada cubeta recibe términos de un único valor de frecuencia. Dentro de cada cubeta todas las claves son iguales, la Inserción no hace desplazamientos ($D = 0$) y el costo es $T(980) = 25(980) - 6(25) + 18 = 24\,368$ operaciones, es decir, $O(n)$.

Bucket Sort no compara elementos para distribuirlos, que es la fase que evita la cota $\Omega(n \log n)$, pero sí usa comparaciones dentro de cada cubeta. Su linealidad depende de que las cubetas sean pequeñas o, como en el corpus, de que cada una contenga claves iguales.

Síntesis del Análisis Teórico

La Tabla 7 resume la complejidad de los tres algoritmos y las condiciones en las que son lineales.

Tabla 7

Complejidad de los Algoritmos y Valores en el Corpus

Algoritmo	Tiempo	Espacio extra	Estable	Condición para ser lineal	Valores en el corpus
Counting Sort	O(n + k)	O(n + k)	Sí	k = O(n)	n = 980, k = 86
Radix Sort (b = 10)	O(d · (n + b))	O(n + b)	Sí	d constante	d = 2, b = 10

Bucket Sort	O(n + m) esperado; $O(n^2)$ peor	O(n + m)	Sí	Claves uniformes o una clave por cubeta	m = 980; 25 cubetas ocupadas
Referencia: por comparación	Ω(n log n)	—	—	—	$og_2(980!) \approx 8$ 300

Implementación

Decisiones de Implementación

Los tres algoritmos usan solo arreglos, ciclos y listas básicas, sin funciones de ordenamiento de la librería estándar en el código de la aplicación; `Arrays.sort` se usa únicamente como oráculo independiente en la batería de pruebas. Cada algoritmo devuelve un arreglo nuevo, de modo que los tres se ejecutan sobre exactamente la misma entrada. Se mueve el objeto `TerminoFrecuencia` completo, no solo la frecuencia, para que cada número siga asociado a su palabra. Finalmente, el orden descendente se obtiene sin invertir el resultado: en Counting y Radix Sort las posiciones de inicio se calculan recorriendo las claves de mayor a menor, y en Bucket Sort las cubetas se concatenan de la mayor a la menor.

Metodología

La clase `MotorRanking` ejecuta los tres algoritmos sobre la lista de frecuencias del corpus completo ($n = 980$) y mide el tiempo real de cada ejecución con el método `System.nanoTime()`, convertido a milisegundos. Para que la medición sea confiable se tomaron cinco precauciones:

1. **Calentamiento de la máquina virtual de Java.** Las primeras ejecuciones de un método corren en modo interpretado, antes de que el compilador JIT lo optimice, y no representan el costo real del algoritmo; por eso se hacen 200 ejecuciones sin cronometrar antes de medir.
2. **Repeticiones intercaladas.** Se mide 200 veces en el orden Counting, Radix, Bucket, Counting, y así sucesivamente, para que ningún algoritmo se vea favorecido o perjudicado por el momento en que corre.
3. **Medición aislada del ordenamiento.** La lista de frecuencias se construye una sola vez, antes de medir, y solo se cronometra la llamada al algoritmo.
4. **Mediana como medida principal.** Algunas repeticiones aisladas se disparan por pausas del recolector de basura; la mediana no se ve afectada por esos valores atípicos, mientras que el promedio sí.
5. **Ejecuciones independientes.** El experimento completo se repitió en cinco procesos distintos de la máquina virtual, para comprobar que el resultado no depende de una ejecución particular.

Resultados sobre el Corpus

La Tabla 8 presenta la mediana del tiempo de cada algoritmo en las cinco ejecuciones independientes, y la Tabla 9 las estadísticas completas de la primera ejecución. La Figura 9 resume los resultados de forma gráfica.

Tabla 8

Algoritmo	Ejec. 1	Ejec. 2	Ejec. 3	Ejec. 4	Ejec. 5	Mediana de las 5
Counting Sort	0.0186	0.0194	0.0260	0.0072	0.0089	0.0186
Radix Sort	0.0381	0.0375	0.0392	0.0378	0.0503	0.0381
Bucket Sort	0.1184	0.0806	0.0876	0.1302	0.1172	0.1172

Fuente: Elaboración propia

Nota. Cada columna corresponde a una ejecución independiente del programa; cada valor es la mediana de 200 repeticiones medidas tras 200 ejecuciones de calentamiento. La última columna es la mediana de las cinco ejecuciones; n = 980 términos.

Tabla 9

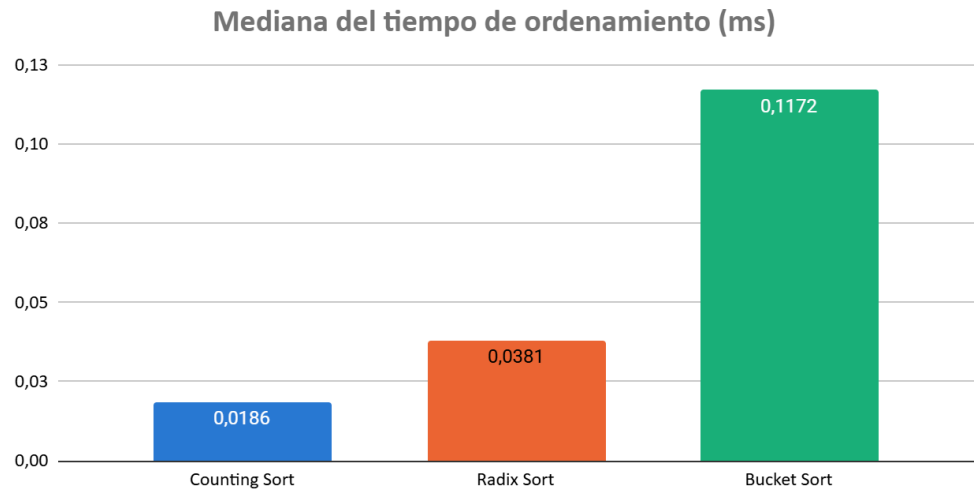
Estadísticas del Tiempo de Ordenamiento en la Primera Ejecución (ms)

Algoritmo	Promedio	Mediana	Mínimo	Máximo	Desviación estándar
Counting Sort	0.0402	0.0186	0.0064	0.2202	0.0463
Radix Sort	0.0475	0.0381	0.0282	0.1860	0.0234
Bucket Sort	0.1578	0.1184	0.0720	0.5480	0.0923

***Fuente**: Elaboración propia*

Figura 9

Tiempo de Ordenamiento de las 980 Frecuencias del Corpus



Fuente: *Elaboración Propia*

Nota. La barra representa la mediana de las cinco ejecuciones

Análisis de los Resultados

Los tres algoritmos ordenan los 980 términos en fracciones de milisegundo, entre 0.01 y 0.13 ms, como corresponde a un costo lineal con un valor de *n* pequeño. El orden Counting Sort < Radix Sort < Bucket Sort se repite en las cinco ejecuciones. Como los tres son O(*n*) sobre este corpus, la diferencia no está en el orden de crecimiento sino en las constantes, es decir, en cuánto trabajo hace cada uno por elemento.

Experimento Complementario de Escalabilidad

Con $n = 980$ los tiempos son tan pequeños que no permiten observar cómo crece el costo. Para estudiar el comportamiento asintótico se generaron listas sintéticas de n términos cuyas frecuencias se sortean con la misma distribución del corpus real ($k \leq 86$), y se midió la mediana de 30 repeticiones para cada n (ver Tabla 10 y Figura 10). Como referencia se incluye `Arrays.sort` de Java, un ordenamiento por comparación que no forma parte del seguimiento y solo sirve para contrastar resultados.

Tabla 10

Mediana del Tiempo de Ordenamiento según el Tamaño de la Entrada (ms).

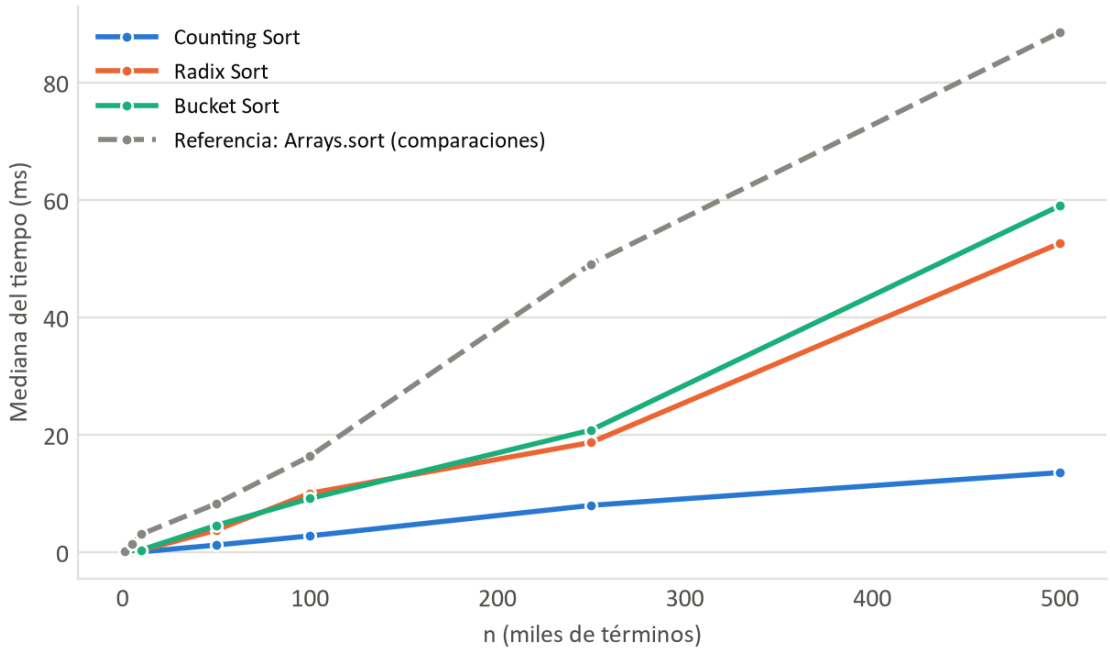
n	Counting Sort	Radix Sort	Bucket Sort	Referencia: Arrays.sort
1 000	0.021	0.141	0.370	0.271
10 000	0.116	0.361	0.444	3.121
50 000	1.276	3.816	4.636	8.332
100 000	2.841	10.098	9.204	16.416
250 000	8.040	18.770	20.862	49.171
500 000	13.624	52.673	59.067	88.574

Fuente: *Elaboración propia*

*L*a Tabla 10 muestra que, con listas de hasta 500 000 términos con la misma distribución del corpus, Counting Sort crece de forma lineal y los tres algoritmos no comparativos son más rápidos que el ordenamiento por comparación de Java, que se incluye solo como referencia.

Figura 10

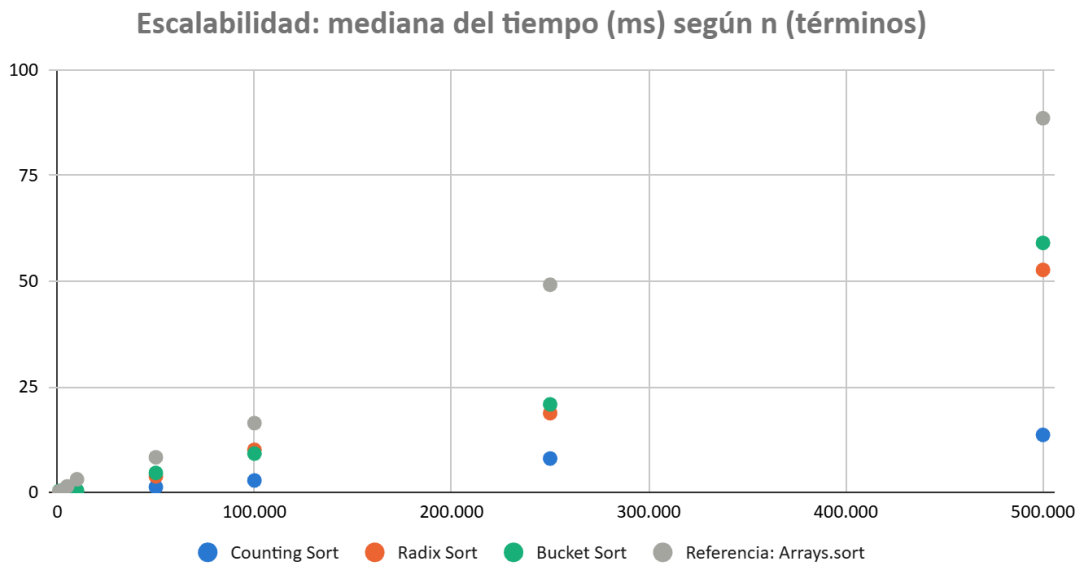
Escalabilidad de los Algoritmos con Listas Sintéticas



Fuente: Elaboración propia

Figura 11

Escalabilidad de los Algoritmos con Listas Sintéticas



Fuente: Elaboración Propia

Counting Sort crece de forma lineal: al pasar de n = 50 000 a n = 500 000, es decir, con diez veces más datos, su tiempo pasa de 1.28 a 13.6 ms, unas 10.7 veces más, como lo predice O(n + k) con k fijo. Radix Sort y Bucket Sort crecen algo más que linealmente en los tamaños grandes (13.8 y 12.7 veces); la causa no es el algoritmo sino la memoria, porque con cientos de miles de elementos los arreglos auxiliares y las listas de cubetas ya no caben en la memoria caché del procesador. Los tres algoritmos no comparativos son más rápidos que el de comparación en todos los tamaños desde n = 10 000; con n = 500 000, Counting Sort es 6.5 veces más rápido que Arrays.sort.

Validación de Resultados

Los tres algoritmos producen exactamente el mismo orden para los 980 términos. La aplicación lo verifica comparando las tres listas posición por posición; se compara el mismo objeto y no solo la frecuencia, de modo que también se verifica el orden de los empates. El resultado se muestra en la interfaz con la etiqueta "Validado", y la batería de pruebas también lo comprueba. La Figura 12 y la Figura 13 presentan los 15 términos más frecuentes obtenidos con cada algoritmo.

Figura 12

Los 15 Términos más Frecuentes del Corpus según cada Algoritmo

Counting Sort

#	Término	Frecuencia
1	ai	86
2	education	57
3	learning	40
4	aied	38
5	educational	33
6	assessment	31
7	intelligence	23
8	artificial	22
9	studies	21
10	higher	16
11	literature	15
12	technologies	15
13	systematic	15
14	future	14
15	systems	14

Radix Sort (LSD, base 10)

#	Término	Frecuencia
1	ai	86
2	education	57
3	learning	40
4	aied	38
5	educational	33
6	assessment	31
7	intelligence	23
8	artificial	22
9	studies	21
10	higher	16
11	literature	15
12	technologies	15
13	systematic	15
14	future	14
15	systems	14

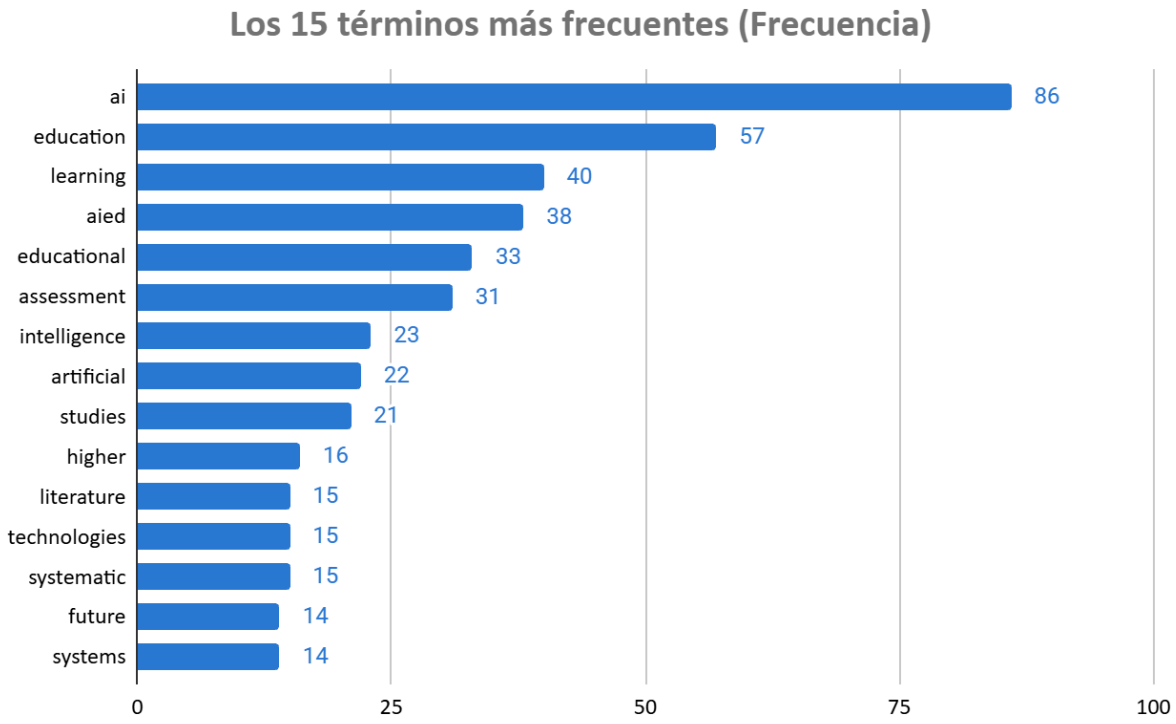
Bucket Sort

#	Término	Frecuencia
1	ai	86
2	education	57
3	learning	40
4	aied	38
5	educational	33
6	assessment	31
7	intelligence	23
8	artificial	22
9	studies	21
10	higher	16
11	literature	15
12	technologies	15
13	systematic	15
14	future	14
15	systems	14

Fuente: *Elaboración propia*

Figura 13

Los 15 Términos más Frecuentes del Corpus



Fuente: Elaboración propia

Tratamiento de los Empates

Como los tres algoritmos son estables y reciben la misma entrada, con los términos en orden de primera aparición, a igual frecuencia aparece primero el término que surgió antes en el corpus. Por ejemplo, *literature*, *technologies* y *systematic* tienen 15 apariciones cada uno, y su orden refleja en qué abstract apareció cada uno por primera vez. El corte del top 15 no es

ambiguo: exactamente dos términos tienen frecuencia 14 (*future* y *systems*) y ocupan los puestos 14 y 15, mientras que el siguiente término, *findings*, tiene frecuencia 13 y queda en el puesto 16.

Interpretación del Ranking

El ranking resume el dominio del corpus. El primer grupo de términos corresponde a la inteligencia artificial: *ai*, *artificial*, *intelligence* y *aied*, sigla de *Artificial Intelligence in Education*. El segundo grupo corresponde a la educación: *education*, *educational*, *learning* y *higher*, este último por la expresión *higher education*. Los términos *studies*, *literature* y *systematic* muestran que la mayoría de los artículos son revisiones sistemáticas de literatura, y la presencia de *assessment*, con 31 apariciones, refleja que varios artículos tratan sobre cómo la inteligencia artificial transforma la evaluación del aprendizaje.

Pruebas de Verificación

Se implementaron pruebas unitarias para cada algoritmo. En lugar de usar JUnit, se escribió una batería propia con la misma estructura (casos de prueba con resultado esperado y aserciones), para mantener el proyecto sin dependencias externas, la batería de pruebas del seguimiento, en el método `probarOrdenamientosNoComparativos`, verifica cada uno de los tres algoritmos. El resultado de cada una de las 33 aserciones y la salida original de la consola se presentan en el Anexo 4: Resultado de Pruebas.

Uso de la Aplicación

La pestaña "Seguimiento · Ranking de términos" de la aplicación web muestra los parámetros del corpus (n, total de palabras, k, d y cubetas ocupadas), la etiqueta "Validado" cuando los tres algoritmos producen el mismo orden, la tabla de tiempos con promedio, mediana, mínimo, máximo y desviación estándar, y los tres top 15 lado a lado. Un selector permite elegir 20, 50 o 200 repeticiones, y el botón "Volver a medir" repite la medición en vivo. Los mismos datos están disponibles en formato JSON en la ruta `GET /api/ranking?top=15&repeticiones=50`, que acepta valores de top entre 1 y 100 y de repeticiones entre 1 y 500.

Conclusiones

Los tres algoritmos no comparativos evitan la cota inferior $\Omega(n \log n)$ porque usan el valor de la clave como índice en lugar de comparar elementos. Las frecuencias de términos son enteros en un rango pequeño, $k = 86$ frente a $n = 980$, que es justo el escenario en el que estos algoritmos son lineales.

Sobre el corpus, Counting Sort es $O(n + k)$ con k mucho menor que n; Radix Sort es $O(n)$ porque $d = 2$ y $b = 10$ son constantes; y Bucket Sort es $O(n)$ porque cada cubeta contiene un único valor de frecuencia, aunque la distribución de Zipf no sea uniforme.

Empíricamente, los tres algoritmos ordenan las 980 frecuencias en menos de 0.13 ms, con el orden Counting < Radix < Bucket en todas las ejecuciones. La diferencia se explica por las constantes, como el número de pasadas sobre los datos y la reserva de memoria, y no por el

orden de crecimiento. El experimento de escalabilidad confirma el crecimiento lineal de Counting Sort y muestra que los tres son más rápidos que un ordenamiento por comparación a partir de $n = 10\,000$.

Finalmente, los tres algoritmos producen exactamente el mismo ranking, incluidos los empates, gracias a su estabilidad. El top 15 refleja el dominio del corpus: la inteligencia artificial aplicada a la educación, con predominio de revisiones sistemáticas y énfasis en la evaluación.

Declaración de Uso de Inteligencia Artificial

Conforme a lo exigido por el curso, se declara que en este seguimiento se usó la herramienta de inteligencia artificial generativa Claude (Anthropic, 2026) como apoyo para la implementación del código fuente.

Anexos

Anexo 1: Graficas del Preprocesamiento

Anexo 2: Arquitectura en capas

Anexo 3: Consolidacion de Resultados de las Pruebas de verificación

Anexo 4: Resultado de Pruebas.

VIGILADA MINEDUCACIÓN

